

TECHNICAL REPORT

Colour Sorting Machine

Using ESP32, TCS34725 RGB Sensor, IR Sensor, MG90S Servo Motor, Automatic Dispensing Mechanism, Conveyor Belt & Cardboard Chassis

Executive Summary

This report presents the complete design, development, and evaluation of a low-cost automated Colour Sorting Machine using plastic bottle caps as the sorted objects. The system employs an ESP32-WROOM-32 microcontroller as the processing unit, integrating a TCS34725 RGB colour sensor and a TCRT5000 infrared (IR) sensor to achieve reliable object detection and colour classification. A JGY-370 high-torque worm gear DC motor drives a conveyor belt that transports caps through the sensing zone continuously — without stopping — while a single MG90S servo motor sweeps non-target covers off the belt into rejection bins.

A key innovation of this build is the automatic cap dispensing mechanism, driven by two additional MG90S servos operating a top gate and bottom gate in sequence. This eliminates manual loading and enables unattended operation. An SSD1306 OLED display provides real-time feedback showing the detection queue, sorting action, and cumulative counts.

The system is designed as a blue-cap sorter: blue caps pass through unobstructed to the end bin, while any other colour is swept off the belt by the servo arm alternating between left and right to balance bin load. Colour classification uses a distance-independent blue ratio algorithm, making detection robust to minor variations in cap distance from the sensor. The cardboard carton chassis provides a zero-cost, reproducible structural platform. This report details the system architecture, hardware selection, circuit design, firmware logic, dispensing mechanism, calibration methodology, challenges encountered, and proposed recommendations.

1. Introduction

1.1 Background and Motivation

Colour sorting is a fundamental process in numerous industrial sectors, ranging from food quality control and pharmaceutical manufacturing to waste recycling and gemstone grading. Traditionally performed manually, it is labour-intensive, slow, and inconsistent. The advent of affordable embedded microcontrollers, precision optical sensors, and programmable actuators has made automated colour sorting practical at a fraction of industrial cost.

This mini-project demonstrates the core principles of automated sorting using an ESP32 microcontroller with off-the-shelf sensors and a cardboard chassis. A primary design goal was simplicity and reliability: the belt never stops, colour classification is continuous, and sorting decisions are queued and executed by the IR sensor trigger at the servo station. This event-driven approach is more robust than a stop-and-sense architecture and scales naturally to higher belt speeds.

1.2 Scope and Limitations

This report covers the complete hardware and firmware design of the colour sorting machine prototype, including the automatic dispensing mechanism and OLED display. The system is designed to operate under controlled indoor lighting with a fixed sensor-to-object distance. Performance under strong variable ambient light is acknowledged in Section 8. The project does not include wireless connectivity in the current version, though this is proposed as a future enhancement.

2. Material and Component Selection

2.1 Colour Sensor

The TCS34725 RGB colour sensor was selected for its 16-bit RGBC output, integrated IR-blocking filter, onboard white LED, and I2C interface. Its wide dynamic range of 3,800,000:1 provides tolerance to minor lighting variations. Classification uses normalised colour ratios rather than raw values, making the algorithm robust to distance variation — a critical requirement since the sensor-to-cap distance on a moving belt cannot be perfectly constant.

2.2 Servo Motor Applications

MG90S metal-gear servo motors are used throughout the system: one for the sorting arm and two for the dispensing mechanism. The MG90S provides adequate torque for sweeping lightweight bottle caps and actuating the dispenser gates, with precise PWM-controlled positioning repeatability better than 2 degrees — sufficient for the bin separations in this design.

2.3 Conveyor Motor

The JGY-370 worm gear DC motor operates at 12V, 150RPM with a rated load current of approximately 300mA and a stall current of 2A. The worm gear provides self-locking behaviour and consistent belt speed. During testing, the motor proved insufficient to drive the original full-width belt under load. The belt width was reduced and the motor was powered from a dedicated external 12V supply, isolated from the ESP32 logic supply, to eliminate power rail noise and ensure adequate current delivery.

2.4 Cardboard Chassis

Corrugated cardboard carton is used for the entire chassis. Its advantages include near-zero cost, ease of fabrication, sufficient rigidity for low-load tabletop operation, and ready availability. All conveyor frames, sensor mounts, bin enclosures, and the dispenser housing are constructed from multi-layer cardboard bonded with hot glue.

3. System Architecture

3.1 High-Level Block Diagram

The system comprises six functional subsystems: the dispensing subsystem, the sensing subsystem, the processing subsystem, the actuation subsystem, the display subsystem, and the power subsystem.

Figure 1 — System Block Diagram: Cap Stack → [Dispenser (Top Servo + Bottom Servo)] → Conveyor Belt → [Colour Sensor (TCS34725)] → ESP32 → [IR Sensor Trigger] → [Sorting Servo → Bins]. OLED Display ← ESP32. External 12V Supply → Belt Motor. ESP32 USB Power → Logic, Sensors, Servos.

3.2 Subsystem Descriptions

3.2.1 Dispensing Subsystem

The automatic dispenser holds a stack of bottle caps in a vertical tube. Two MG90S servos control a top gate and a bottom gate. On each dispense cycle: the top gate closes to lock the upper stack, the bottom gate opens to release the lowest cap onto the belt, the bottom gate closes, then the top gate opens to allow the next cap to descend. This four-step sequence delivers one cap every 2.5 seconds for reliable single-cap feeding.

3.2.2 Sensing Subsystem

The TCS34725 RGB sensor is mounted at a fixed height above the belt and reads caps continuously as they pass underneath. The sensor's onboard white LED illuminates each cap. Colour ratios are calculated from the 16-bit RGBC output and compared against calibrated thresholds. The TCRT5000 IR sensor is mounted at the servo station downstream — when a cap interrupts the IR beam, the ESP32 executes the next colour decision from the queue.

3.2.3 Processing Subsystem — Queue Architecture

The ESP32 runs a continuous queue-based firmware. The colour sensor feeds detections into a first-in-first-out queue of up to 10 entries. When the IR sensor fires, the next entry is dequeued and the sorting servo executes the corresponding action. This decoupled architecture means the belt never needs to stop, and multiple caps can be in transit simultaneously without confusion.

3.2.4 Actuation Subsystem

Three MG90S servos are employed. The sorting servo rests at 90 degrees home position. For non-blue caps it alternates between sweeping left (145 degrees) and sweeping right (35 degrees) to balance load between two rejection bins. Blue caps trigger no servo movement and ride the belt to the end bin. The two dispenser servos operate the top and bottom gates of the cap feeder.

3.2.5 Display Subsystem

An SSD1306 0.96-inch OLED display shares the I2C bus with the TCS34725 colour sensor on GPIO25 (SDA) and GPIO26 (SCL). The display shows a queue grid of up to 10 labelled boxes (B for blue, R for red, G for green, X for unknown), an IR trigger status line, and cumulative counters for blue and non-blue caps processed.

3.2.6 Power Subsystem

The ESP32 is powered via USB. The belt motor is powered from a dedicated external 12V supply to prevent motor transients from affecting the logic rail. Servo motors are powered from the 5V external supply rail. All grounds are joined at a common reference point. The TCS34725 and OLED are powered from the ESP32 3.3V pin.

4. Hardware Components

4.1 Component Summary Table

Component	Function	Interface	Key Specification
-----------	----------	-----------	-------------------

ESP32-WROOM-32	Microcontroller	GPIO, I2C, PWM	240MHz, 4MB Flash, 520KB SRAM
TCS34725	RGB Colour Sensor	I2C (0x29)	16-bit RGBC, 3,800,000:1 dynamic range
TCRT5000	IR Proximity Sensor	Digital GPIO	25mm range, 5V, NPN output
MG90S (x3)	Sorting + Dispenser Servos	PWM (50Hz)	180°, 2.2kg·cm torque, metal gear
JGY-370 DC Motor	Conveyor Belt Drive	Direct (external 12V)	12V, 150RPM, 300mA rated, 2A stall
SSD1306 OLED	Real-time Display	I2C (0x3C)	128x64px, monochrome, 0.96 inch
1N4007 Diode	Motor Flyback Protection	Passive	1A, 1000V reverse voltage
Cardboard Carton	Chassis / Frame	Structural	Multi-layer corrugated, free-form

Table 1: Complete Bill of Materials with Technical Specifications

5. Circuit Design

5.1 GPIO Pin Assignment

ESP32 Pin	Function	Connected To
GPIO 25 (SDA)	I2C Data	TCS34725 SDA + OLED SDA (shared bus)
GPIO 26 (SCL)	I2C Clock	TCS34725 SCL + OLED SCL (shared bus)
GPIO 34	Digital Input (input-only)	TCRT5000 IR Sensor OUT
GPIO 13	PWM Output	MG90S Sorting Servo Signal
GPIO 14	PWM Output	MG90S Top Dispenser Servo Signal
GPIO 27	PWM Output	MG90S Bottom Dispenser Servo Signal
3.3V Pin	Power	TCS34725 VIN, OLED VCC
5V Pin	Power	TCRT5000 VCC, Servo VCC (external)
GND	Common Ground	All modules, external supply GND

Table 2: ESP32 GPIO Pin Assignment

Note: GPIO25 and GPIO26 were assigned as the I2C bus pins rather than the ESP32 default GPIO21/GPIO22, following practical verification during build. Both the TCS34725 (address 0x29) and SSD1306 OLED (address 0x3C) share this bus without conflict. GPIO34 is used for the IR sensor as it is an input-only pin, which is appropriate for sensor signal reception.

5.2 Motor Wiring

The JGY-370 belt motor is connected directly to a dedicated external 12V supply via a 1N4007 flyback diode across the motor terminals (cathode to positive, anode to negative). The motor is always on — there is no switching circuit — consistent with the design decision to never stop the belt. The external supply ground is joined to the ESP32 common ground rail.

5.3 Servo Wiring

All three MG90S servos draw signal from their respective ESP32 GPIO pins. VCC (red wire) is connected to the external 5V supply rail, not the ESP32 5V pin, to prevent servo current draw from resetting the board. All servo GND (brown wire) connects to the common ground rail shared with the ESP32.

6. Firmware and Software Design

6.1 Development Environment

Firmware is written in C++ using the Arduino framework for ESP32, developed in Arduino IDE 2.x. The following libraries are used:

- Wire.h — I2C communication
- Adafruit_TCS34725.h — TCS34725 colour sensor driver
- ESP32Servo.h — PWM servo control for ESP32
- Adafruit_SSD1306.h — OLED display driver
- Adafruit_GFX.h — Graphics primitives for OLED

6.2 Firmware Architecture — Queue-Based Sorting

The firmware implements a queue-based event-driven architecture. The colour sensor reads continuously and enqueues each new detection. The IR sensor at the servo station acts as a dequeue trigger: when a cap arrives, the next queued colour decision is executed. This fully decouples detection from sorting and allows multiple caps to be in transit on the belt simultaneously.

Key design rules enforced in firmware: a cap is only enqueued when the sensor transitions from no-object (clear channel below threshold) to object-detected, preventing the same cap from being enqueued multiple times as it passes under the sensor. After each detection, a gap must be confirmed before the next cap can be queued.

6.3 Colour Classification Algorithm

Classification uses a blue ratio calculated from the normalised RGB channels:

```
blueRatio = B / (R + G + B)
If blueRatio >= 0.42 → BLUE (pass through)
If blueRatio < 0.42 → NOT BLUE (sweep)
```

This ratio-based approach is distance-independent: if the cap is slightly closer or farther from the sensor, all three channels scale proportionally and the ratio remains stable. Calibration measurements confirmed a clear separation: blue caps produce a ratio of 0.41–0.51, while all other colours and ambient readings produce ratios below 0.35.

Condition	Norm R	Norm G	Norm B	Blue Ratio
Ambient (no cap)	135–138	70–71	43–45	~0.17 (rejected)
Blue Cap	40–45	78–83	125–131	0.41–0.51 (BLUE)
Green Cap	69–71	111–120	49–61	~0.21 (NOT BLUE)
Red Cap	169	51	31	~0.12 (NOT BLUE)

Table 3: Calibrated Colour Ratio Measurements

6.4 Sorting Logic

Blue caps trigger no servo movement — they ride the belt to the end bin. Non-blue caps trigger the sorting servo, which alternates between left (145°) and right (35°) on each successive non-blue cap. This alternating sweep distributes rejected caps evenly between two side bins without the servo needing to return to a full home position between consecutive non-blue caps, improving throughput.

6.5 Dispensing Mechanism Firmware

The dispenser runs on a timer: every 2,500ms, the `dispenseCap()` function executes a four-step sequence. Step 1: top servo closes (150°) to lock the stack. Step 2: bottom servo opens (90°) to release the lowest cap. Step 3: bottom servo closes (0°). Step 4: top servo opens (75°) to allow the next cap to descend. Delays between steps ensure mechanical completion before the next command is issued.

6.6 OLED Display

The SSD1306 OLED is updated after every queue change and IR trigger event. The display layout consists of a title bar, a 10-box queue grid (B = blue, R = red, G = green, X = unknown, empty box = vacant slot), an IR status line showing the last executed action, and a stats bar showing cumulative blue count, non-blue count, and current queue depth.

7. Calibration and Testing

7.1 Colour Sensor Calibration

Calibration was performed with the sensor mounted at its final fixed operating height above the belt. Each cap colour was held stationary under the sensor and multiple readings were taken. Ambient light readings were also recorded. The blue ratio threshold of 0.42 was selected to provide a margin of at least 0.07 above the maximum non-blue ratio (0.35) and at least 0.06 below the minimum blue ratio (0.41), providing reliable separation under typical indoor lighting.

7.2 Servo Calibration

All three servo angle positions were verified by observation. The sorting servo home (90°), left sweep (145°), and right sweep (35°) positions were tuned to ensure clean cap diversion without mechanical interference with the belt edges. Dispenser servo open and closed positions were tuned to reliably hold and release single caps without jamming.

7.3 Belt and Dispenser Timing

The dispense interval of 2,500ms was set to ensure adequate spacing between consecutive caps on the belt — sufficient time for the colour sensor to detect and queue each cap before the next arrives. The queue architecture means that even if timing varies slightly, the system remains accurate as long as caps are detected and queued before reaching the IR sensor.

8. Challenges Encountered and Recommendations

8.1 Belt Motor Insufficient for Full-Width Belt

The JGY-370 motor at 12V and 150RPM proved unable to drive the original full-width conveyor belt under load. The motor would stall or significantly slow when caps were placed on the belt, causing detection timing errors. Two measures resolved this: the belt width was reduced to decrease the load on the motor, and the motor was powered from a dedicated external 12V supply separated from the ESP32 logic supply. Powering the motor from the same supply as the ESP32 caused voltage rail noise and occasional ESP32 resets.

Recommendation: for a wider or faster belt, a higher-torque motor or a motor driver with PWM speed control would provide better performance. At minimum, the motor supply should always be isolated from the microcontroller supply.

8.2 I2C Pin Mapping

The default ESP32 I2C pins (GPIO21, GPIO22) did not function correctly on the NodeMCU ESP32E board variant used in this build. After systematic debugging using an I2C scanner sketch, the I2C bus was reassigned to GPIO25 (SDA) and GPIO26 (SCL), which resolved all communication issues with both the TCS34725 and OLED. The root cause was a pin layout discrepancy between the standard ESP32 Dev Board diagram (30-pin) and the actual 38-pin NodeMCU ESP32E board used.

Recommendation: always run an I2C scanner sketch to verify sensor addresses and confirm bus operation before integrating sensor-specific library code. Do not assume pin positions from generic diagrams — verify against the physical board's actual 38-pin layout.

8.3 Colour Sensor False Detections (BXXBXX Pattern)

Early firmware produced a repeating BLUE-NOTBLUE-BLUE-NOTBLUE pattern in the queue even with only blue caps present. Investigation revealed two causes: the sensor was catching the trailing edge of a departing cap as a new NOT_BLUE detection, and the MIN_CLEAR threshold was set too high, causing some covers to be missed entirely. The fix was implementing gap detection logic: the firmware only accepts a new colour detection after the sensor has confirmed a clear gap (clear channel below threshold) between covers, preventing edge reads from being treated as new objects.

Recommendation: for any conveyor colour sensing application, always implement gap detection rather than relying solely on time-based lockouts. The physical gap between objects is a more reliable signal than any fixed delay.

8.4 Ambient Light Interference

Strong overhead fluorescent lighting caused the TCS34725 clear channel readings to increase significantly, compressing the ratio difference between colours. A cardboard shroud around the sensor mounting area reduced ambient light infiltration and stabilised readings. With the shroud in place and constant sensor height, the blue ratio separation became reliable and consistent.

Recommendation: shield the colour sensor from ambient light at the design stage. Even a simple tube or hood around the sensor and the object reading zone dramatically improves classification reliability.

8.5 Sensor Distance Variation

Initial testing with the sensor at variable heights produced fluctuating readings. The blue channel value varied between approximately 100 and 131 depending on cap distance, which risked misclassification. Switching from absolute threshold detection to ratio-based detection resolved this, as ratios remain stable across the tested distance range. Fixing the sensor height mechanically further improved consistency.

Recommendation: use ratio-based colour classification rather than absolute channel values for any application where object-to-sensor distance cannot be guaranteed to be perfectly constant.

8.6 Upload and Board Configuration Issues

Initial firmware uploads produced MD5 mismatch and serial noise errors. These were resolved by holding the BOOT button during the upload sequence, reducing the upload baud rate from 921600 to 115200, and ensuring the correct board (ESP32 Dev Module) was selected in the Arduino IDE. The board package was installed via manual hotspot download due to slow institutional network speeds.

Recommendation: for ESP32 development, always set upload speed to 115200 for reliable uploads, and confirm the board package is the official Espressif ESP32 package.

9. Results and Discussion

9.1 System Performance

The assembled system demonstrates reliable blue cap sorting with the belt running continuously. The queue-based architecture allows multiple caps to be in transit simultaneously without sorting errors. The OLED display provides clear real-time feedback on system state. The automatic dispenser feeds caps at a consistent 2.5-second interval, enabling fully unattended operation during demonstration.

The alternating servo sweep mechanism effectively distributes non-blue caps between the two side bins. No mechanical jamming of the dispenser or sorting arm was observed during testing under normal conditions.

9.2 Limitations of the Current Design

- System is calibrated for the specific operating lighting and sensor height — recalibration required if these change.
- Belt motor is sensitive to load — heavy or wide belts may require a higher-torque motor or motor driver.
- Cardboard chassis is not suitable for humid, outdoor, or high-vibration environments.
- The dispenser is designed for uniform cylindrical bottle caps — irregular objects may jam.
- No error recovery for jammed or stuck objects on the belt.

10. Areas of Application

10.1 Waste Management and Recycling

The most direct real-world application of this system is in plastic cap sorting for recycling. Bottle caps are produced in large quantities and are typically sorted by colour for recycling grade classification. A scaled version of this machine could process hundreds of caps per minute in a recycling facility.

10.2 Food and Beverage Industry

Colour sorting is widely used in food processing to grade produce by ripeness and quality. The ratio-based classification approach used in this project is directly analogous to the spectral ratio methods used in commercial food colour sorters.

10.3 Pharmaceutical Manufacturing

Tablet and capsule colour verification is a regulatory requirement in pharmaceutical production. The automatic dispensing mechanism demonstrated in this project mirrors the single-unit feeding systems used in tablet inspection machines.

10.4 Education and Research

This prototype serves as an accessible hands-on platform for teaching embedded systems, sensor integration, mechatronics, queue-based algorithms, and system-level design. The low cost and reproducibility of the cardboard chassis make it deployable in school and university laboratories with minimal budget.

11. Future Enhancements

11.1 Machine Learning Classification

The current ratio threshold classifier is effective for the tested colours but would struggle with novel or similar hues. A TensorFlow Lite model deployed on the ESP32 could classify a much wider range of colours with greater robustness. Training data can be collected using the existing calibration sketch.

11.2 IoT Dashboard Integration

The ESP32 WiFi capability enables real-time sorting data streaming to a cloud platform. A web dashboard could display per-colour counts, throughput, and error rates, with remote threshold adjustment.

11.3 Improved Chassis and Motor

Replacing the cardboard chassis with 3D-printed or aluminium extrusion components would improve structural rigidity, reduce vibration, and enable higher belt speeds. A motor driver with PWM speed control would allow belt speed to be adjusted in firmware.

11.4 Multi-Colour Sorting Expansion

The queue architecture and alternating servo design can be extended to multi-colour sorting by adding a second servo arm or a rotating disc diverter, enabling three or more colour categories to be sorted into separate bins simultaneously.

12. Conclusion

This report has presented the comprehensive design, construction, calibration, and testing of a colour sorting machine built using an ESP32-WROOM-32 microcontroller, TCS34725 RGB sensor, TCRT5000 infrared sensor, three MG90S servo motors, a JGY-370 worm gear DC motor, an SSD1306 OLED display, and a cardboard carton chassis. The system demonstrates automated blue cap sorting using a continuous belt, queue-based detection, and an automatic cap dispensing mechanism — all operating without manual intervention.

Key technical contributions of this build include the development of a distance-independent ratio-based colour classification algorithm, a gap-detection approach that eliminates false queue entries, an alternating servo sweep that balances bin load, and a four-step dispenser mechanism enabling unattended operation. Significant challenges in board pin mapping, motor power isolation, ambient light shielding, and firmware upload configuration were encountered and resolved, with lessons documented for future builds.

The system's underlying principles are directly applicable to recycling, food processing, pharmaceutical manufacturing, and education. The proposed future enhancements provide a clear roadmap for evolving the prototype toward an industrially relevant platform.

References

- [1] Espressif Systems. (2023). ESP32 Series Datasheet v3.4.
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [2] ams AG. (2015). TCS34725 Colour Light-to-Digital Converter with IR Filter Datasheet. ams AG.
- [3] Vishay Semiconductors. (2008). TCRT5000 Reflective Optical Sensor Datasheet. Vishay Intertechnology.

- [4] Chen, X., et al. (2018). Colour-Based Object Sorting Using TCS3200 and Arduino. *International Journal of Engineering and Applied Sciences*, 5(4), 112–118.
- [5] Adafruit Industries. (2023). Adafruit TCS34725 Color Sensor Library. https://github.com/adafruit/Adafruit_TCS34725
- [6] Adafruit Industries. (2023). Adafruit SSD1306 OLED Library. https://github.com/adafruit/Adafruit_SSD1306
- [7] Singh, R., & Kumar, A. (2020). Embedded System Based Automated Colour Sorting Machine. *IEEE ICAEECI*, 1–6.
- [8] TensorFlow. (2024). TensorFlow Lite for Microcontrollers. <https://www.tensorflow.org/lite/microcontrollers>

Appendix A: Cost of Materials

Component	Qty	Unit Cost (NGN)	Total Cost (NGN)
ESP32-WROOM-32 Dev Board	1	8630	8630
USB 5V to 12V Boost Converter	1	3270	3270
12V to 3.3,5,12V Power module	1	2550	2550
TCS34725 RGB Sensor Module	1	6920	6920
TCRT5000 IR Sensor Module	1	3010	3010
MG90S Servo Motor	3	4540	13620
JGY-370 DC Gear Motor	1	5770	5770
SSD1306 OLED Display	1	4210	4210
1N4007 Flyback Diode	1	30	30
Conveyor Belt / Rubber Band	1	0	0
Cardboard Carton	1	0	0
Miscellaneous (wire, glue, caps)	1	1500	1500
TOTAL	—	—	49510

Table A1: Estimated Bill of Materials (market prices, approx.)

Note: All components are available from online electronics suppliers (e.g. Temu, Jumia). Prices vary by supplier and region. The cardboard chassis had zero material cost as recycled packaging cartons were used. The VL53L0X Time-of-Flight sensor originally specified was removed from the final design as its detection role was fulfilled by the IR sensor and colour sensor working together, reducing both cost and complexity.